WEBX IA 1 AB

Q1. Define Web Analytics 2.0.

Definition: Analysis of digital data from multiple online sources for business insights.

- Analyzes clickstream, social media, and video data.
- Focuses on customer experience and multi-channel behavior.
- Aims to optimize marketing campaigns and website conversions.

Q2. Define Clickstream Analysis.

Definition: The process of tracking and analyzing a user's website navigation path.

- Reveals popular pages, entry/exit points, and traffic patterns.
- Helps understand user flow and potential website bottlenecks.
- Provides data for improving site navigation and content placement.

Q3. Characteristics of Semantic Web.

- **Standardized Formats:** Uses W3C standards like RDF and OWL for data representation.
- **Data Integration:** Enables linking and merging data from diverse sources seamlessly.
- **Intelligent Reasoning:** Allows machines to infer new facts from existing relationships.
- **Machine-Readable:** Focuses on creating data that can be processed directly by computers.

Q4. Define `RDF`.

Definition: Resource Description Framework, a standard model for data interchange on the web.

- Represents information as statements in subject-predicate-object form (triples).
- Uses URIs to uniquely identify resources and their relationships.
- Creates a directed, labeled graph structure for linking arbitrary data.

Q5. Define `SPARQL`.

Definition: SPARQL Protocol and RDF Query Language, used to query RDF data.

- Performs complex joins across diverse datasets in triple stores.
- Supports queries for graph patterns, conjunctions, and disjunctions.
- Functions as a key technology for accessing the Semantic Web.

Q6. What is `TypeScript`?

Definition: A strongly typed programming language that builds on JavaScript.

- Compiles to plain JavaScript for execution anywhere.
- Adds optional static types, classes, and modules to JavaScript.
- Developed and maintained by Microsoft for large-scale applications.

Q7. Advantages of `TypeScript`.

- **Static Typing:** Catches errors during development, improving code quality.
- **Enhanced IDE Support:** Provides superior autocomplete and refactoring capabilities.
- **Improved Readability:** Types act as inline documentation for complex code.
- **ECMAScript Compatibility:** Supports latest JavaScript features and compiles to older versions.

Q8. Define `TypeScript` modules.

Definition: Self-contained pieces of code that execute in their own scope, not the global scope.

- Variables, functions, and classes inside a module are private by default.
- Use the `export` keyword to make module members accessible externally.
- Use the `import` keyword to consume exported members from other modules.

Q9. What is `AngularJS`?

Definition: A structural JavaScript framework for building dynamic web applications.

- Uses HTML as a template language with extended directives.
- Implements MVC (Model-View-Controller) architecture on the client-side.
- Facilitates two-way data binding between the model and view components.

Q10. Define data binding.

Definition: The automatic synchronization of data between the model and view components.

- **Two-way binding:** Changes in the UI update the model, and vice-versa.
- **One-way binding:** Data flows only from the model to the view.
- Reduces boilerplate code needed for DOM manipulation and event handling.

Q11. What is dependency injection?

Definition: A design pattern where components receive their dependencies from an external source.

- Promotes loose coupling between a class and its dependencies.
- Enhances testability by allowing mock dependencies to be injected easily.
- `AngularJS` uses this pattern to provide components like services and factories.

Q12. What is `ng-repeat`?

Definition: An `AngularJS` directive that iterates over a collection and clones HTML elements.

- Instantiates a template once per item from an array or object.
- Provides special variables like `$index`, `$first`, and `$last` within the loop.
- Updates the DOM automatically when the underlying collection changes.

Q13. What is `$scope`?

Definition: An `AngularJS` object that acts as the glue between the controller and the view.

- Contains the model data and functions accessible to the view template.
- Watches for model changes to update the view via data binding.
- Follows a prototypal inheritance hierarchy within nested controllers.

Q14. Define Semantic Web Stack.

Definition: A layered architecture of technologies and standards for the Semantic Web.

- **Base Layer:** Unicode, URI, and XML provide the foundation for syntax and identification.
- **Data Layer:** RDF and RDF Schema model and describe relationships between resources.
- **Ontology Layer:** OWL (Web Ontology Language) defines complex concepts and relationships.
- **Query Layer:** SPARQL is used to retrieve and manipulate data across the stack.

Q15. Explain evolution of WebX.0 with diagram.

Definition: Progression of the internet from static pages to intelligent, connected ecosystems.

- **Web 1.0 (Read-Only):** Static content, information portals, and read-only websites.
- **Web 2.0 (Read-Write):** Interactive platforms, social media, user-generated content, and collaboration.
- **Web 3.0 (Executable):** Semantic web, AI-driven, decentralized applications, and personalized experiences.
- **Web 4.0 (Intelligent):** Symbiotic web, ubiquitous connectivity, and human-AI interaction.

Diagram:

```
Interactivity & Intelligence
                ^
```

```
                |
     Web 4.0 -----+-----> Intelligent Web
      (2020+)    |        (AI, Symbiotic)
                 |
     Web 3.0 -----+-----> Semantic Web
      (2010+)    |        (AI, RDF, Blockchain)
                 |
     Web 2.0 -----+-----> Social Web
      (2000s)    |        (Interaction, UGC)
                 |
     Web 1.0 -----+-----> Read-Only Web
      (1990s)    |        (Static, Information)
                 |
                 +------------------> Time
```

Q16. Explain Web Analytics 2.0 model.

Definition: Web Analytics 2.0 measures, analyzes, and optimizes online data to improve digital business outcomes comprehensively.

- **Clickstream Data:** Analyzes user path and behavior on the website for navigation insights.
- **Outcomes Analysis:** Measures multi-channel conversions and success metrics beyond just page views.
- **Experimentation:** Tests various hypotheses through `A/B` and multivariate testing methodologies.
- **Voice of Customer:** Collects direct user feedback through surveys and sentiment analysis.
- **Competitive Intelligence:** Benchmarks website performance against market competitors and industry trends.

Q17. Draw and explain Semantic Web Stack.

Definition: The Semantic Web Stack is a layered architecture enabling machine-understandable data through standardized technologies.

- **Unicode & URI:** Provides international character set and unique resource identification foundation.

- **XML & Namespaces:** Offers surface syntax for structured documents with element differentiation.
- **RDF & RDF Schema:** Creates basic data model for resources and defines vocabulary relationships.
- **Ontology (OWL):** Adds formal semantics for more complex knowledge representation and reasoning.
- **Logic & Proof:** Enables intelligent deductions and validates the authenticity of semantic statements.
- **Trust:** Establishes confidence in the information through digital signatures and verification.

Diagram:

```
+---------------------+
|       Trust         |
+---------------------+
|       Proof         |
+---------------------+
|      Logic          |
+---------------------+
|     Ontology OWL    |
+---------------------+
|    RDF + RDF Schema |
+---------------------+
|    XML + NS + XML   |
|       Schema        |
+---------------------+
|   Unicode  |  URI   |
+------------+--------+
```

Q18. Explain `RDF` and `SPARQL` with example.

Definition: RDF models web data as statements; SPARQL is the query language to retrieve that data.

- **RDF Triple:** Consists of subject-predicate-object structure to declare a fact.
- **RDF Example:** "The sky has the color blue" formatted as a subject-predicate-object triple.
- **SPARQL Protocol:** Standardized interface for querying RDF datasets across the web.

- **SPARQL Query:** Similar to `SQL` but designed for graph patterns to fetch RDF data.

Example:

- **RDF Triple:** `<http://example.org/sky> <http://example.org/hasColor> "blue".`
- **SPARQL Query:**

```
SELECT ?object
WHERE {
  <http://example.org/sky> <http://example.org/hasColor> ?object .
}
```

Q19. Explain `TypeScript` internal architecture.

Definition: `TypeScript` architecture is a multi-stage compiler pipeline transforming TS code to JS.

- **Scanner:** Breaks source code into a stream of fundamental syntax tokens.
- **Parser:** Organizes tokens into an Abstract Syntax Tree representing code structure.
- **Binder:** Creates symbols and connects declarations to form a symbol table.
- **Type Checker:** Validates code correctness based on inferred and defined types.
- **Transformer:** Converts `TypeScript AST` to `JavaScript AST` for compatibility.
- **Emitter:** Generates the final `JavaScript` output along with definition files.

Q20. Explain `TypeScript` data types with examples.

Definition: `TypeScript` data types enforce variable value types for safer, predictable code.

- **Static Typing:** Allows type declaration at compile time to prevent runtime errors.
- **Type Inference:** Automatically deduces variable type based on assigned value.

Types (list each type and one point of it):

- `number` : Represents both integer and floating-point numeric values.
- `string` : Stores textual data enclosed within single or double quotes.
- `boolean` : Represents logical true or false values for conditions.

- `array` : Defines a collection of elements with a specific type.
- `tuple` : Fixed-length array where each element has a particular type.
- `enum` : Set of named constants providing more readable code.
- `any` : Disables type checking for a variable, allowing any value.
- `void` : Absence of any type, commonly used for function return.
- `null/undefined` : Represents intentional absence of value or uninitialized variable.
- `never` : Represents values that never occur, like function throwing exceptions.

Example:

```
let isDone: boolean = false;
let decimal: number = 6;
let color: string = "blue";
let list: number[] = [1, 2, 3];
let x: [string, number] = ["hello", 10];
```

Q21. Explain `TypeScript` classes and inheritance.

Definition: Classes are blueprints for objects; inheritance derives new classes from existing ones.

- **Class Members:** Includes properties, constructors, and methods defined within a class.
- **Constructor:** Special method for creating and initializing class object instances.
- **Access Modifiers:** Controls member visibility using `public`, `private`, or `protected` keywords.
- **Inheritance:** Uses `extends` keyword to create subclass from a parent class.
- **Super Keyword:** Calls parent class constructor or methods from derived class.
- **Method Overriding:** Allows subclass to provide specific implementation of parent method.

Example:

```
class Animal {
  name: string;
  constructor(name: string) { this.name = name; }
  move(): void { console.log(`${this.name} moves`); }
}
```

```
class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    super(name);
    this.breed = breed;
  }
  move(): void {
    super.move();
    console.log(`${this.name} runs`);
  }
}
```

Q22. Explain `TypeScript` modules with example.

Definition: Modules organize code into separate files with their own scope, not globally.

- **Export Keyword:** Makes functions, classes, or variables available outside the module.
- **Import Keyword:** Brings exported declarations from another module into current file.
- **Module Loader:** Specifies module resolution strategy like `CommonJS` or `AMD`.
- **Encapsulation:** Keeps module internals private, preventing namespace pollution globally.

Example:

```
// module1.ts
export function greet(name: string): string {
  return `Hello, ${name}`;
}
export const PI = 3.14;

// module2.ts
import { greet, PI } from './module1';
console.log(greet('World'));
console.log('Value of PI:', PI);
```

Q23. Explain `AngularJS` architecture.

Definition: `AngularJS` is a MVC framework for building dynamic, single-page web applications.

- **MVC Pattern:** Separates application logic into Model, View, and Controller components.
- **Scope:** Acts as glue between controller and view, holding model data.
- **View:** Represents the `DOM` loaded and rendered by the `AngularJS` compiler.
- **Controller:** Contains `JavaScript` functions to set up and manipulate scope data.
- **Model:** Derived from scope attributes containing the application's data state.
- **Two-way Binding:** Synchronizes data automatically between model and view components.

Q24. Explain two-way data binding in `AngularJS`.

Definition: Two-way binding synchronizes data between model and view automatically in real-time.

- **View to Model:** User input updates the model data automatically without code.
- **Model to View:** Programmatic data changes reflect instantly in the user interface.
- **$watch:** Registers listeners to detect changes in scope variables constantly.
- **$apply:** Manually triggers digest cycle when changes happen outside Angular context.
- **Digest Cycle:** Loops through all watchers to check and update data changes.
- **ng-model:** Directive enabling two-way binding specifically for form input elements.

Example:

```
<div ng-app="" ng-controller="myCtrl">
  Name: <input ng-model="name">
  <h1>You entered: {{name}}</h1>
</div>
<script>
function myCtrl($scope) { $scope.name = 'John Doe'; }
</script>
```

Q25. Explain `AngularJS` directives with example.

Definition: Directives extend HTML with custom attributes and elements to build dynamic UIs.

- **ng-app:** Auto-bootstraps `AngularJS` application defining the root element.

- **ng-controller:** Attaches a controller class to the view template section.
- **ng-repeat:** Instantiates a template once per item from a collection.
- **ng-model:** Binds input, select, and textarea values to scope properties.
- **ng-click:** Specifies custom behavior when an element is clicked.
- **ng-show/ng-hide:** Conditionally shows or hides elements based on expression.

Example:

```
<div ng-app="" ng-controller="myCtrl">
  <button ng-click="toggle()">Click Me</button>
  <p ng-show="showMessage">Hello World!</p>
</div>
<script>
function myCtrl($scope) {
  $scope.showMessage = false;
  $scope.toggle = function() { $scope.showMessage =
!$scope.showMessage; };
}
</script>
```

Q26. Explain dependency injection in `AngularJS`.

Definition: Dependency injection supplies components with their required dependencies rather than hard-coding them.

- **Injector Service:** Locates and provides dependencies to components requesting them.
- **Singleton Pattern:** Maintains single instance of service throughout application lifecycle.
- **Controller Injection:** Dependencies passed as arguments to controller constructor function.
- **Minification Safe:** Uses array annotation or $inject property to preserve names.

Example:

```
// Normal Injection
function myCtrl($scope, $http) {
  $http.get('url').then(...);
}
```

```
// Minification Safe Array Annotation
myCtrl = ['$scope', '$http', function($scope, $http) {
  $http.get('url').then( ... );
}];
```

Q27. Explain `AngularJS` services.

Definition: Services are singleton objects that provide reusable business logic across application components.

- **Lazy Instantiation:** Services are only created when a component depends on them.
- **Factory Method:** Returns an object containing the service's functionality and properties.
- **Service Method:** Instantiates a constructor function to create the service object.
- **Provider Method:** Configurable service creation with setup phase before application runs.
- **$http Service:** Facilitates communication with remote `HTTP` servers via `AJAX` calls.
- **$log Service:** Provides logging capabilities to debug and monitor applications.
  Built-in Services Examples:
- **$http:** For making `AJAX` calls to web servers.
- **$location:** For parsing `URL` in browser address bar.
- **$timeout:** For wrapping `window.setTimeout` function.
- **$filter:** For formatting data displayed to users.

Q28. Explain routing using `ngRoute`.

Definition: ngRoute enables deep-linking and navigation between different views in single-page applications.

- **$routeProvider:** Configures routes mapping URLs to specific views and controllers.
- **when() Method:** Defines a new route with path, template, and controller parameters.
- **otherwise() Method:** Sets the default redirect route for unrecognized paths.
- **ng-view Directive:** Container where routed template content is rendered in layout.
- **$routeParams:** Retrieves current route's parameter values from the URL.

Example:

```
// Configuration
app.config(function($routeProvider) {
  $routeProvider
  .when('/home', { templateUrl: 'home.html', controller:
'homeCtrl' })
  .when('/about', { templateUrl: 'about.html', controller:
'aboutCtrl' })
  .otherwise({ redirectTo: '/home' });
});

// HTML
<div ng-view></div>
```

Q29. Explain form validation in `AngularJS`.

Definition: `AngularJS` form validation provides real-time feedback on user input based on defined rules.

- **Novalidate:** Disables browser's native validation to use `AngularJS` validation.
- **Form State:** Tracks form states like pristine, dirty, valid, and invalid.
- **Field State:** Monitors individual input field states for targeted feedback.
- **ng-minlength/ng-maxlength:** Validates input length against minimum or maximum values.
- **ng-pattern:** Ensures input matches a specific regular expression pattern.
- **ng-required:** Marks field as mandatory and invalid if left empty.
  CSS Classes:
- **ng-valid:** Applied when form or field passes validation rules.
- **ng-invalid:** Applied when form or field fails validation rules.
- **ng-pristine:** Applied when field has not been modified by user.
- **ng-dirty:** Applied when field has been modified by user.
- **ng-touched:** Applied when field has lost focus.

Example:

```
<form name="myForm" novalidate>
  <input name="email" type="email" ng-model="user.email" required>
  <span ng-show="myForm.email.$dirty &&
```

```
myForm.email.$invalid">Invalid Email</span>
</form>
```